TECHNICAL ASSIGNMENT SUBMISSION

Music Catalogue & Daily Featured Artist Service

Technical Assignment Submission & Architecture Documentation

PREPARED FOR

Lars Gentsch
Technical Hiring Manager

Dila Yapar
Human Resources

SUBMITTED BY

Shakil Jansberg
Candidate

POSITION	DATE	REPOSITORY
Senior Backend Engineer	06.10.2026	`ICE_Assignment_Architecture_2`

Contents

1. Assignment Objective & Problem Statement 1

2. Architecture & Design Principles 2

2.1 Architectural Pattern: Modular Monolith with DDD Bounded Contexts 2

2.2 Key Architectural Benefits 2

2.3 Cloud Target Architecture (AWS Production Scaling Blueprint) 3

3. Domain Modelling & Algorithmic Solutions 4

3.1 Requirement 1: Track Attribution & Catalogue Integrity 4

3.2 Requirement 2: The Auto-Alias Preservation Pattern 4

3.3 Requirement 3: Enriched Discography Fetching & Filtering 5

3.4 Requirement 4: Cyclical "Artist of the Day" Engine 5

4. API Specifications & Contracts 7

4.1 Endpoints Overview 7

4.2 Endpoint Details & Contract Examples 7

5. Testing Strategy & Verification Evidence 10

5.1 Testing Strategy Overview 10

5.2 Test Suites Breakdown 10

5.3 Test Execution Log 10

6. Security, Resilience & DevSecOps Implementation 12

6.1 Authentication & Role-Based Access Control (RBAC) 12

6.2 Observability & Distributed Tracing 12

6.3 Secure Containerization 12

7. Frontend: Developer Admin Console 13

8. Step-by-Step Execution Guide 14

8.1 System Prerequisites 14

8.2 Option A: One-Click Local Execution (Recommended) 14

8.3 Option B: Manual Local Startup 14

8.4 Option C: Docker Compose Execution 14

8.5 Verification & Inspection URLs 15

8.6 Quick curl Smoke Test Commands 15

9. Conclusion 16

1. Assignment Objective & Problem Statement

The objective of this assignment is to design and implement a production-ready music metadata service for a global streaming platform. The platform manages artists, their pseudonyms and aliases, track discographies, and an automated promotional feature on the homepage.

The assignment defines four core functional requirements:

1. **Add a New Track to an Artist's Catalogue.** Users can add a new track to an existing artist's discography, capturing attributes such as track title, genre, duration, and industry-standard metadata.
2. **Edit Artist Name & Accommodate Aliases.** Users can update an artist's canonical name while retaining and managing multiple stage names, alter-egos, or legal aliases without corrupting historical catalogue attributions.
3. **Fetch Artist Tracks.** Users can retrieve all tracks associated with a specific artist, with support for filtering and structured metadata presentation.
4. **Fair, Cyclical "Artist of the Day".** The homepage features a different artist each day in a fair, cyclical rotation across the entire catalogue. If there are N active artists, every artist must be featured exactly once over N days, after which the rotation restarts with the first artist.

The submission also has to be production-ready. That covers clean domain modelling, algorithmic correctness, efficiency under read-heavy traffic, security, testability, and containerized deployment.

2. Architecture & Design Principles

2.1 Architectural Pattern: Modular Monolith with DDD Bounded Contexts

For a service of this scope, splitting the system into a multi-repo or multi-service deployment (four separate Spring Boot services with Eureka, an API gateway, and distributed message brokers) adds operational fragility:

- Multi-process memory overhead (more than 4 GB RAM) and slow container startup.
- Distributed transaction failures and two-phase commit overhead on a domain with three entities.
- Evaluation risk from network port conflicts or inter-service race conditions.

To balance pragmatism with room to scale, the solution is built as a **Modular Monolith** that follows strict **Domain-Driven Design (DDD)** bounded contexts.

```
com.ice.music
├── artist                  <-- Bounded Context: Artist Identity & Alias Preservation
│   ├── controller          <-- REST Endpoints (/api/v1/artists)
│   ├── dto                 <-- Request/Response DTOs & Validation
│   ├── model               <-- JPA Entities (Artist, ArtistAlias)
│   ├── repository          <-- Spring Data Repositories
│   └── service             <-- Business Logic & Auto-Alias Archiving
├── track                   <-- Bounded Context: Catalogue & Discography Management
│   ├── controller          <-- REST Endpoints (/api/v1/tracks, /api/v1/artists/{id}/tracks)
│   ├── dto                 <-- Track DTOs, Discography Summary DTO
│   ├── model               <-- Track JPA Entity
│   ├── repository          <-- Track Repository with Query Indexing
│   └── service             <-- Validation, Formatting & Filtering Logic
├── artistofday             <-- Bounded Context: Cyclical Rotation & Homepage Promotion
│   ├── controller          <-- REST Endpoints (/api/v1/featured/today)
│   ├── dto                 <-- Featured Artist & Countdown DTOs
│   └── service             <-- Stateless Cyclical Rotation Engine & Caching
└── shared                  <-- Cross-Cutting Infrastructure
    ├── config              <-- OpenAPI/Swagger & Caffeine In-Memory Cache
    ├── exception           <-- Global Exception Handler (RFC 7807 Problem Details)
    └── security            <-- Stateless JWT, DevSecOps Controls, MDC Tracing
```

2.2 Key Architectural Benefits

1. **Independent Bounded Contexts.** Modules communicate only through public service interfaces and immutable DTO records. No module touches another module's internal JPA repository or table directly.
2. **Low Evaluation Friction.** The system compiles in seconds, runs all unit and integration tests in under 15 seconds, and starts with a single command or Docker Compose.
3. **Straightforward Cloud Extraction.** If traffic grows, each package (`artist`, `track`, `artistofday`) can be moved into its own AWS ECS Fargate container without rewriting domain logic.

2.3 Cloud Target Architecture (AWS Production Scaling Blueprint)

In a high-throughput production environment managing tens of millions of tracks and rightsholders, the modular monolith would evolve into the following AWS infrastructure:

- **Relational Core (Postgres RDS):** the ACID system of record for rights management, artist identities, and track catalogue entries.
- **Search & Filter Scaling (Amazon OpenSearch):** for queries across 50M+ tracks, an asynchronous event stream populates an OpenSearch index. Discography full-text search and multi-genre facets are served from that index, with a target latency under 10 ms.
- **Artist of the Day Cache (Amazon DynamoDB Global Tables):** daily featured artists are pre-computed and stored in DynamoDB with automated Time-To-Live (TTL), serving reads in under 1 ms to global CloudFront edge caches.
- **Asynchronous Integration (Transactional Outbox to Amazon Kinesis / SQS):** cross-domain notifications (for example, telling royalty distribution services about artist alias changes) use the Transactional Outbox pattern to publish domain events reliably to Kinesis without distributed two-phase commits.

3. Domain Modelling & Algorithmic Solutions

3.1 Requirement 1: Track Attribution & Catalogue Integrity

In the music industry, tracks are copyright-protected intellectual property tied to legal rightsholders. A track cannot exist as an orphan; it must belong to a verified artist.

- **Referential Verification:** before persisting a new track, `TrackService` verifies that `artistId` exists in the database. If it does not, the service responds immediately with an RFC 7807 compliant `404 Not Found` instead of letting a database foreign-key violation bubble up.
- **Formatted Duration:** the backend accepts `durationMs` (strictly greater than 0) and computes a `durationFormatted` string in `mm:ss` format (for example, 215000 ms becomes `03:35`). This avoids repeating the calculation on every client device.
- **Database Indexing:** the `tracks` table has B-Tree indexes on `artist_id` (`idx_tracks_artist_id`) and `genre` (`idx_tracks_genre`), giving O(log M) discography lookups.

3.2 Requirement 2: The Auto-Alias Preservation Pattern

A common anti-pattern in catalogue systems is treating an artist name change as a plain `UPDATE artists SET name = :newName`. In the music copyright ecosystem this causes serious data loss:

- Artists perform under stage names, alter-egos, and legal names (for example, Marshall Mathers ↔ Eminem ↔ Slim Shady, or Donald Glover ↔ Childish Gambino ↔ mcDJ).
- Overwriting the old name erases historical attribution. Past releases and royalty contracts registered under the former name can no longer be resolved.
- If tracks reference artist names as mutable strings instead of immutable UUIDs, renaming an artist breaks every track relationship.

Implementation in ArtistService

1. **Immutable UUID Identity.** The artist's identity is an unchangeable UUID (`id`). All tracks link strictly to `artist_id`.
2. **Automatic History Archival.** When an administrator changes an artist's canonical name from A to B, the service first archives name A into the `artist_aliases` table, then sets the new name to B.
3. **Explicit Alias Appending.** Any new aliases in the payload are deduplicated and appended to the artist's alias list.
4. **Optimistic Locking (`@Version`).** The `Artist` entity uses JPA `@Version` concurrency control. If two administrators edit the same artist at the same time, the second write is rejected with HTTP 409 Conflict, which prevents lost updates.

5. **Cache Eviction.** Updating an artist invalidates the "Artist of the Day" cache so the data stays consistent.

3.3 Requirement 3: Enriched Discography Fetching & Filtering

- **Query Flexibility:** an artist's discography can be fetched with an optional genre filter (`GET /api/v1/artists/{id}/tracks?genre=Hip-Hop`).
- **Enriched Summary Response:** the endpoint returns an `ArtistDiscographyResponse` containing:
  - The artist's canonical name and full alias history.
  - The total number of tracks in the catalogue for that artist.
  - A distinct list of all genres present in the artist's work, used to populate frontend filter chips.
  - The list of tracks, sorted by release/creation date, newest first.

3.4 Requirement 4: Cyclical "Artist of the Day" Engine

The requirement states:

> *"As a user, you should be able to see a different 'Artist of the Day' in a cyclical manner on the homepage each day, ensuring a fair rotation through the entire catalogue of artists. This means if there are n artists, after n days, the cycle restarts with the first artist, ensuring an equal chance for each artist to be the 'Artist of the Day'."*

Why Naive Approaches Fail in Production

- **`ORDER BY RAND() LIMIT 1`:** non-deterministic and non-cyclical. An artist could appear three days in a row while others are never featured.
- **Nightly cron job or scheduler:** if an instance restarts or crashes at midnight UTC, that day's rotation is missed. In clustered deployments, cron jobs cause race conditions unless distributed locking is added.
- **DB column `last_featured_at`:** creates write contention. Every homepage hit would either need a write lock or periodic database updates on read-heavy traffic.
- **Modulo on database IDs `(epoch % max_id)`:** IDs have gaps from deletions. ID 5 might not exist, which would leave blank days.

The Deterministic Modulo Formula

The rotation engine (`CyclicalRotationEngine`) is stateless and deterministic:

```
epochDay = LocalDate.now(ZoneOffset.UTC).toEpochDay()
targetIndex = Math.floorMod(epochDay, N)
```

Where:

- `epochDay` is the day count since the Unix epoch. It increases by 1 at every midnight UTC.
- `N` is the total count of active artists in the catalogue (`countByActiveTrue()`).
- `Math.floorMod` handles both positive and negative epoch values safely.

Stable Total Ordering

To guarantee that index k maps to the same artist on every cluster node and read replica, the query enforces a strict compound sort order:

```
SELECT a FROM Artist a
WHERE a.active = true
ORDER BY a.createdAt ASC, a.id ASC
LIMIT 1 OFFSET :targetIndex;
```

Proof of Fairness & Cycle Invariance

1. **Fairness.** For any N consecutive calendar days D, D+1, ..., D+N-1, the modulo formula produces the set {floorMod(D+k, N) for k from 0 to N-1}, which equals {0, 1, 2, ..., N-1}. Every active artist is featured exactly once during the N-day period.
2. **Cycle Restart.** On day D+N, floorMod(D+N, N) equals floorMod(D, N). The rotation restarts with the same artist who started the cycle on day D.
3. **No Distributed Race Conditions.** Because the result depends only on the calendar date and the total count, every instance resolves the same featured artist at the same time with no inter-service coordination.

Caching

- **Caffeine In-Memory Cache:** the featured artist is cached under the key `YYYY-MM-DD`.
  - First hit of the day: one indexed query (under 3 ms).
  - Every later homepage hit: an in-memory O(1) lookup (under 0.02 ms).
- **Live UTC Countdown:** the API returns `secondsRemainingInDay` until midnight UTC (00:00:00 UTC), which drives a real-time countdown clock in the frontend.

4. API Specifications & Contracts

4.1 Endpoints Overview

Method	Path	Access	Description
`GET`	`/api/v1/featured/today`	Public	Returns the current Artist of the Day with UTC countdown
`GET`	`/api/v1/artists`	Public	Returns all active artists with their aliases
`GET`	`/api/v1/artists/{id}`	Public	Returns details and aliases for a specific artist
`POST`	`/api/v1/artists`	Editor	Creates a new artist with optional initial aliases
`PUT`	`/api/v1/artists/{id}`	Editor	Updates artist canonical name (auto-preserves old name as alias)
`GET`	`/api/v1/artists/{id}/tracks`	Public	Returns artist discography (supports `?genre=...` filtering)
`POST`	`/api/v1/tracks`	Editor	Adds a new track to an artist's catalogue
`GET`	`/api/v1/tracks`	Public	Returns all tracks across the platform
`POST`	`/api/v1/auth/dev-token`	Public	Generates a valid Bearer token for API evaluation

4.2 Endpoint Details & Contract Examples

1. Featured Artist of the Day

- **URL:** `GET /api/v1/featured/today`
- **Response (HTTP 200 OK):**

```
{
  "artistId": "da256a27-025b-4eca-963a-d7a2ea9b1ae5",
  "canonicalName": "Childish Gambino",
  "aliases": ["Donald Glover", "mcDJ"],
  "featuredDate": "2026-10-05",
  "cycleDayIndex": 1,
  "totalActiveArtists": 5,
  "secondsRemainingInDay": 22745,
  "sampleTracks": [
    {
      "id": "e4c9b3a1-7788-4e12-b91a-982134567890",
      "title": "Redbone",
      "genre": "Funk",
      "durationMs": 326000,
      "durationFormatted": "05:26",
      "isrc": "USGL11600001"
    }
  ]
}
```

2. Add New Track to Artist Catalogue

- **URL:** `POST /api/v1/tracks`
- **Headers:** `Authorization: Bearer <TOKEN>`, `Content-Type: application/json`
- **Request:**

```
{
  "artistId": "da256a27-025b-4eca-963a-d7a2ea9b1ae5",
  "title": "Sober",
  "genre": "R&B",
  "durationMs": 252000,
  "isrc": "USGL11400042"
}
```

- **Response (HTTP 201 Created):**

```
{
  "id": "f5a8b1c2-3d4e-4f5a-8b9c-0d1e2f3a4b5c",
  "artistId": "da256a27-025b-4eca-963a-d7a2ea9b1ae5",
  "title": "Sober",
  "genre": "R&B",
  "durationMs": 252000,
  "durationFormatted": "04:12",
  "isrc": "USGL11400042",
  "createdAt": "2026-10-05T15:45:00Z"
}
```

3. Edit Artist Name & Manage Aliases

- **URL:** `PUT /api/v1/artists/{id}`
- **Headers:** `Authorization: Bearer <TOKEN>`, `Content-Type: application/json`
- **Request:**

```
{
  "canonicalName": "Marshall Mathers",
  "newAliases": ["Slim Shady"]
}
```

- **Response (HTTP 200 OK):**

```
{
  "id": "c1a2b3c4-d5e6-4f7a-8b9c-0d1e2f3a4b5c",
  "canonicalName": "Marshall Mathers",
  "aliases": ["Eminem", "Slim Shady"],
  "version": 2,
  "updatedAt": "2026-10-05T15:46:12Z"
}
```

Note: the former canonical name "Eminem" was preserved in `aliases` automatically, without manual re-entry.

4. Fetch Artist Tracks (With Optional Genre Filter)

- **URL:** `GET /api/v1/artists/{id}/tracks?genre=Funk`

- **Response (HTTP 200 OK):**


```
{
  "artistId": "da256a27-025b-4eca-963a-d7a2ea9b1ae5",
  "artistName": "Childish Gambino",
  "aliases": ["Donald Glover", "mcDJ"],
  "totalTracks": 1,
  "availableGenres": ["Funk", "R&B"],
  "tracks": [
    {
      "id": "e4c9b3a1-7788-4e12-b91a-982134567890",
      "title": "Redbone",
      "genre": "Funk",
      "durationMs": 326000,
      "durationFormatted": "05:26",
      "isrc": "USGL11600001",
      "createdAt": "2026-10-01T08:00:00Z"
    }
  ]
}
```


5. Standardized Error Handling (RFC 7807)

When validation fails or a resource is missing, the API responds in the Problem Details format:


```
{
  "type": "about:blank",
  "title": "Resource Not Found",
  "status": 404,
  "detail": "Artist not found with ID: 99999999-9999-9999-9999-999999999999",
  "instance": "/api/v1/tracks",
  "timestamp": "2026-10-05T15:47:00Z"
}
```

5. Testing Strategy & Verification Evidence

5.1 Testing Strategy Overview

The test suite is built for fast execution and to avoid flaky tests:

1. **Unit Tests (Mockito & AssertJ):** cover pure business rules and mathematical invariants (rotation correctness, alias archiving, duration calculation, negative epoch safety).
2. **Integration Tests (Spring MockMvc):** cover the web and security layers, checking HTTP status codes, JSON serialization, and role-based access control.
3. **Database Migrations (Flyway on in-memory H2):** every test run executes against a freshly migrated database, which verifies that the schema creation and seed data scripts are syntactically valid.

5.2 Test Suites Breakdown

Test Suite Class	Tests Run	Target Under Test	Key Assertions
`CyclicalRotationEngineTest`	3	Deterministic Rotation Algorithm	Day N+1 resets to day 1; handles N=0 (empty catalogue); negative epoch days do not cause index errors.
`ArtistServiceTest`	3	Artist Service & Auto-Alias Rule	Initial alias creation; auto-archiving of the former canonical name on rename; 404 when the ID does not exist.
`TrackServiceTest`	2	Track Service & Validation	Track creation and duration formatting (`05:26`); rejection of orphan tracks when the artist is missing.
`SecurityIntegrationTest`	3	Spring Security & JWT Filters	`/api/v1/featured/today` is publicly accessible; write endpoints return 403 Forbidden for anonymous requests; write endpoints succeed for `ROLE_EDITOR`.
Total	**11**	**Full System Validation**	**All 11 tests pass**

5.3 Test Execution Log

The log below was produced by running the test suite through the Maven wrapper:

```
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< com.ice:ice-music-platform >--------------------
[INFO] Building ICE Music Platform 1.0.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-resources-plugin:3.3.1:resources (default-resources) ---
```

```
[INFO] Copying 3 resources from src\main\resources to target\classes
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-resources-plugin:3.3.1:testResources (default-testResources) ---
[INFO] skip non existing resourceDirectory src\test\resources
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:testCompile (default-testCompile) ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-surefire-plugin:3.2.5:test (default-test) ---
[INFO] Running com.ice.music.artist.ArtistServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.814 s -- in
com.ice.music.artist.ArtistServiceTest
[INFO] Running com.ice.music.artistofday.CyclicalRotationEngineTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.052 s -- in
com.ice.music.artistofday.CyclicalRotationEngineTest
[INFO] Running com.ice.music.security.SecurityIntegrationTest
2026-10-05T17:41:25.075+02:00  INFO 45856 --- [ice-music-platform] [main] o.f.core.internal.command.DbValidate :
Successfully validated 2 migrations
2026-10-05T17:41:25.126+02:00  INFO 45856 --- [ice-music-platform] [main] o.f.core.internal.command.DbMigrate  : Migrating
schema "PUBLIC" to version "1 - init schema"
2026-10-05T17:41:25.155+02:00  INFO 45856 --- [ice-music-platform] [main] o.f.core.internal.command.DbMigrate  : Migrating
schema "PUBLIC" to version "2 - seed data"
2026-10-05T17:41:25.175+02:00  INFO 45856 --- [ice-music-platform] [main] o.f.core.internal.command.DbMigrate  :
Successfully applied 2 migrations to schema "PUBLIC", now at version v2
2026-10-05T17:41:29.567+02:00  INFO 45856 --- [ice-music-platform] [main] c.i.m.security.SecurityIntegrationTest : Started
SecurityIntegrationTest in 7.896 seconds
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 8.670 s -- in
com.ice.music.security.SecurityIntegrationTest
[INFO] Running com.ice.music.track.TrackServiceTest
2026-10-05T17:41:29.985+02:00  INFO 45856 --- [ice-music-platform] [main] c.ice.music.track.service.TrackService : Adding
track 'Redbone' for artist id da256a27-025b-4eca-963a-d7a2ea9b1ae5
2026-10-05T17:41:29.990+02:00  INFO 45856 --- [ice-music-platform] [main] c.ice.music.track.service.TrackService : Adding
track 'Orphan Track' for artist id ad0a6c53-5674-4051-9ab2-cbfaf9d2eafc
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.092 s -- in com.ice.music.track.TrackServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  13.340 s
[INFO] Finished at: 2026-10-05T17:41:30+02:00
[INFO] ------------------------------------------------------------------------
```

6. Security, Resilience & DevSecOps Implementation

6.1 Authentication & Role-Based Access Control (RBAC)

- **Stateless JWT:** standard Bearer token authentication through the Spring Security `SecurityFilterChain`.
- **Least Privilege Access:**
  - Read endpoints (`GET /api/v1/featured/today`, `/api/v1/artists/**`, `/api/v1/tracks/**`) are open to public consumers.
  - Modification endpoints (`POST /api/v1/artists`, `PUT /api/v1/artists/**`, `POST /api/v1/tracks`) require `ROLE_EDITOR`.
- **Reviewer Helper:** the `/api/v1/auth/dev-token` endpoint issues a valid JWT signed with the internal key. The frontend includes a one-click "Editor Mode (JWT)" button that attaches this token automatically.

6.2 Observability & Distributed Tracing

- **MDC Correlation IDs:** every inbound HTTP request passes through `CorrelationIdFilter`, which reads the `X-Correlation-ID` header (or generates a UUID if it is absent) and attaches it to the SLF4J Mapped Diagnostic Context (MDC). All later log messages include `[correlationId=...]`.
- **Spring Boot Actuator:** exposes operational endpoints:
  - `/actuator/health`: reports liveness, database connectivity, and disk status.
  - `/actuator/info`: exposes build and application metadata.

6.3 Secure Containerization

The `Dockerfile` uses a multi-stage build and runs under an unprivileged user:

```
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser:appgroup
```

Running as a non-root user limits the impact of a container compromise and follows the CIS Docker Benchmark recommendation.

7. Frontend: Developer Admin Console

The frontend is built with **Angular 18** using standalone components, with a deliberately plain **Developer Admin Console** design:

- **Functional by design:** it is an internal operations tool rather than a consumer app. It has dense tables, clear action buttons, modal dialogs, and instant visual feedback.
- **Reviewer-friendly workflow:** Editor Mode is enabled on initialization, so track creation and artist editing can be tested immediately without a separate login portal.
- **Visible algorithm state:** the top status panel shows the current cycle day, the total count of active artists, the index formula (epochDay mod N), and a live countdown to midnight UTC.
- **Immediate verification:** changing an artist's canonical name updates the table at once, shows the previous name in the aliases column, and increments the entity version.

8. Step-by-Step Execution Guide

8.1 System Prerequisites

- **Java:** JDK 21 or JDK 25.
- **Node.js:** Node 18+ and npm (for the frontend).
- **Docker & Docker Compose:** optional, for containerized execution.

8.2 Option A: One-Click Local Execution (Recommended)

A local execution script is included in the project root:

```
cd ICE_Assignment_Architecture_2
.\run-local.bat
```

The script does the following:

1. Verifies the Java and Node.js environments.
2. Builds and starts the Spring Boot backend on `http://localhost:8080`.
3. Installs frontend dependencies and launches the Angular application on `http://localhost:4200`.

8.3 Option B: Manual Local Startup

Step 1: Start the Backend

```
cd ICE_Assignment_Architecture_2\backend
.\mvnw.cmd spring-boot:run
```

- The backend starts on `http://localhost:8080`.
- The in-memory H2 database is initialized and seeded by Flyway with 5 artists and their tracks.

Step 2: Start the Frontend

```
cd ICE_Assignment_Architecture_2\frontend
npm install
npm start
```

- Open `http://localhost:4200` in a modern web browser.

8.4 Option C: Docker Compose Execution

To run the full multi-container environment (PostgreSQL, Spring Boot backend, and NGINX frontend):

```
cd ICE_Assignment_Architecture_2
docker compose up --build
```

8.5 Verification & Inspection URLs

- **Frontend Admin Console:** `http://localhost:4200`
- **Swagger UI / OpenAPI Documentation:** `http://localhost:8080/swagger-ui.html`
- **Actuator Health Probe:** `http://localhost:8080/actuator/health`
- **H2 Database Console (Local Profile):** `http://localhost:8080/h2-console`
  - JDBC URL: `jdbc:h2:mem:ice_music_db`
  - Username: `sa`
  - Password: leave blank

8.6 Quick curl Smoke Test Commands

```
# 1. Fetch the Artist of the Day
curl -X GET "http://localhost:8080/api/v1/featured/today"

# 2. Generate a Dev JWT Bearer Token
TOKEN=$(curl -s -X POST "http://localhost:8080/api/v1/auth/dev-token" | grep -o '"token":"[^"]*' |
cut -d'"' -f4)

# 3. Add a New Track to an Artist
curl -X POST "http://localhost:8080/api/v1/tracks" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "artistId": "da256a27-025b-4eca-963a-d7a2ea9b1ae5",
    "title": "Feels Like Summer",
    "genre": "R&B",
    "durationMs": 297000
  }'

# 4. Fetch Tracks for that Artist
curl -X GET "http://localhost:8080/api/v1/artists/da256a27-025b-4eca-963a-d7a2ea9b1ae5/tracks"
```

9. Conclusion

This submission covers all four requirements in the assignment brief. The Artist of the Day rotation is a deterministic modulo calculation over a stable ordering, artist renames preserve the old name as an alias, and the bounded contexts, automated tests, and containerized setup are in place to support running it as a production service.